Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial

Optimasi Penataan Kendaraan pada Kapal
Menggunakan Local Search



Kelompok 5:

18224038	Christian Wilfredo Pakpahan
18224098	Nicholas Putra Halim
18224118	Brandon Zeko Alexander

Dosen Pengampu :
Dr. Nur Ulfa Maulidevi, ST, M.Sc.

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

DAFTAR ISI **2**
BAB 1 ..... **3**
DESKRIPSI PERSOALAN **3**
Deskripsi **3**
BAB 2 ..... **4**
PEMBAHASAN **4**
2.1. Objective Function 4
2.2. Implementasi Algoritma Local Search 5
a. Hill-Climbing Steepest Ascent 10
b. Simulated Annealing 17
c. Genetic Algorithm 23
BAB 3 ..... **28**
HASIL DAN ANALISIS **28**
a. Steepest Ascent Hill-Climbing 28
b. Simulated Annealing 31
c. Genetic Algorithm 36
d. Analisis 42
BAB 4 ..... **45**
KESIMPULAN DAN SARAN **45**
4.1. Kesimpulan 45
4.2. Saran 45
PEMBAGIAN TUGAS **47**
REFERENSI **48**
LAMPIRAN FORM PENGGUNAAN GENERATIVE AI **49**

BAB 1
DESKRIPSI PERSOALAN

Deskripsi



Anda adalah seorang *Forward Deployed Engineer* (FDE) di PT ~~Owen~~ Intim Line yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda.

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik dan keputusan pemilihan letak kendaraan belum tentu menghasilkan keuntungan yang optimal.

Tim operasional meminta Anda membangun sistem pendukung keputusan untuk menentukan kendaraan yang akan diangkut serta posisi dan orientasi setiap kendaraan pada geladak. Sistem tersebut harus dapat mengevaluasi berbagai kemungkinan konfigurasi, mempertimbangkan fungsi-fungsi objektif yang ditentukan, dan menghasilkan solusi penataan yang optimal.

BAB 2
PEMBAHASAN

2.1. Objective Function

Fungsi objektif (*objective function*) dirancang untuk memaksimalkan total keuntungan pengiriman (*shipping fee*) dari kendaraan yang berhasil dimuat ke dalam kapal, sekaligus meminimalkan pelanggaran terhadap batasan (*constraints)* fisik kapal. Kapal memiliki dimensi area 10 x 16 grid unit dengan batas beban maksimum sebesar 50 unit.

Secara matematis, nilai fungsi objektif $f(S)$ untuk suatu konfigurrasi *state S* dihitung menggunakan persamaan:

$$f(S) = \sum_{v \in V_{loaded}} Fee\,(v) - (P_{overlap} + P_{boundary} + P_{weight})$$

Penalti diberikan secara tegas untuk memastikan solusi akhir bernilai valid dan realistis secara fisik. Denda tumpang tindih (*overlap penalty)* dikenakan jika terdapat sel kendaraan yang saling menindih di dalam kapal:

$$P_{overlap} = 500\ x\ N_{overlap\ cells}$$

Denda luar batas (boundary penalty) dikenakan jika terdapat bagian kendaraan yang keluar dari batas grid kapal 10 × 16:

$$P_{boundary} = 500\ x\ N_{out\ cells}$$

Denda kelebihan beban (*excess weight penalty*) dikenakan untuk setiap unit kelebihan berat di atas kapasitas maksimum 50 unit:

$$P_{weight} = 200\ x\ max(0,\ W_{total} - 50)$$

Suatu state dinyatakan valid apabila seluruh penalti bernilai nol ($P_{overlap} = P_{boundary} = P_{weight} = 0$), sehingga nilai objektif yang diperoleh merupakan keuntungan murni dari biaya pengiriman kendaraan.

Sebaliknya, jika *state* melanggar salah satu batasan fisik tersebut, besarnya nilai penalti akan mendominasi perhitungan sehingga menghasilkan nilai objektif yang sangat negatif pada iterasi-iterasi awal.

2.2. Implementasi Algoritma Local Search

entities.py

```
class Vehicle:
    def __init__(self, id, width, length, shipping_fee, weight, eta):
        self.id = id
        self.width = width
        self.length = length
        self.shipping_fee = shipping_fee
        self.weight = weight
        self.eta = eta

class Ship:
    def __init__(self, width, length, max_capacity):
        self.width = width
        self.length = length
        self.max_capacity = max_capacity

def load_problem_from_dict(data):
    ship_data = data["ship"]
    ship = Ship(ship_data["width"], ship_data["length"], ship_data["max_capacity"])

    vehicles = []
    for v in data["vehicles"]:
        veh = Vehicle(v["id"], v["width"], v["length"], v["shipping_fee"], v["weight"], v["eta"])
        vehicles.append(veh)

    return ship, vehicles
```

File entities.py perannya lebih ke pendefinisian struktur data dari entitas permasalahan optimasi tata letak kendaraan pada kapal. Di dalamnya bisa dilihat ada kelas vehicle yang tugasnya merepresentasikan setiap kandidat muatan beserta atribut bawaannya (seperti dimensi, berat, dan nilai keuntungan), serta kelas ship yang mendefinisikan constraint lingkungan pencarian berupa ukuran geladak dan kapasitas maksimal kapal. Bagian yang paling penting di sini adalah fungsi load_problem_from_dict() yang bertugas mem-parsing data mentah konfigurasi awal menjadi objek yang terstruktur. Misalnya dari yang awalnya JSON hanya teks angka untuk V1 (lebarnya 3, panjang 4, dan shipping fee 60) langsung diubahnya jadi objek Vehicle utuh di memori.

state.py

```
import copy
import random

class ShipState:
```

```
def __init__(self, ship, vehicles, placements=None):
    self.ship = ship
    self.vehicles = vehicles

    # Kalau placements belum ada, langsung buat random (initial state)
    if placements is None:
        self.placements = self.generate_random_placements()
    else:
        self.placements = placements

def generate_random_placements(self):
    placements = []
    for v in self.vehicles:
        is_loaded = random.choice([True, False])
        orientation = random.choice(["Horizontal", "Vertical"])

        if orientation == "Horizontal":
            eff_w = v.width
            eff_l = v.length
        else:
            eff_w = v.length
            eff_l = v.width

        max_x = max(0, self.ship.width - eff_w)
        max_y = max(0, self.ship.length - eff_l)

        x = random.randint(0, max_x)
        y = random.randint(0, max_y)

        placements.append({
            "is_loaded": is_loaded,
            "x": x,
            "y": y,
            "orientation": orientation
        })
    return placements

def copy_state(self):
    new_placements = copy.deepcopy(self.placements)
    return ShipState(self.ship, self.vehicles, new_placements)

def get_vehicle_size(self, index):
    v = self.vehicles[index]
    ori = self.placements[index]["orientation"]
    if ori == "Horizontal":
        return v.width, v.length
    else:
        return v.length, v.width

def get_objective_value(self):
```

```
    total_fee = 0
    total_weight = 0
    overlap_count = 0
    out_of_bounds_count = 0

    # Buat grid 2D ukuran kapal, isi awal 0 semua
    grid = []
    for r in range(self.ship.length):
        row = [0] * self.ship.width
        grid.append(row)

    for i in range(len(self.vehicles)):
        p = self.placements[i]
        if not p["is_loaded"]:
            continue

        v = self.vehicles[i]
        total_fee += v.shipping_fee
        total_weight += v.weight

        w, l = self.get_vehicle_size(i)

        for dx in range(w):
            for dy in range(l):
                gx = p["x"] + dx
                gy = p["y"] + dy

                # Cek apakah keluar batas geladak kapal
                if gx < 0 or gx >= self.ship.width or gy < 0 or gy >= self.ship.length:
                    out_of_bounds_count += 1
                else:
                    # Cek apakah cell grid overlapping
                    if grid[gy][gx] >= 1:
                        overlap_count += 1
                    grid[gy][gx] += 1

    # Cek apakah melebihi kapasitas berat kapal
    excess_weight = 0
    if total_weight > self.ship.max_capacity:
        excess_weight = total_weight - self.ship.max_capacity

    # Kurangi total fee dengan penalti jika melanggar constraint
    penalty = (overlap_count * 500) + (out_of_bounds_count * 500) + (excess_weight * 200)
    return total_fee - penalty

def is_valid(self):
    total_weight = 0
    grid = []
    for r in range(self.ship.length):
        grid.append([0] * self.ship.width)
```

```
    for i in range(len(self.vehicles)):
        p = self.placements[i]
        if not p["is_loaded"]:
            continue

        v = self.vehicles[i]
        total_weight += v.weight
        if total_weight > self.ship.max_capacity:
            return False

        w, l = self.get_vehicle_size(i)
        for dx in range(w):
            for dy in range(l):
                gx = p["x"] + dx
                gy = p["y"] + dy
                if gx < 0 or gx >= self.ship.width or gy < 0 or gy >= self.ship.length:
                    return False
                if grid[gy][gx] >= 1:
                    return False
                grid[gy][gx] += 1
    return True

def get_random_neighbor(self):
    neighbor = self.copy_state()
    n = len(neighbor.placements)
    if n == 0:
        return neighbor

    # 3 operasi neighbor sesuai spesifikasi
    move_type = random.choice(["swap", "move", "rotate"])

    if move_type == "swap" and n >= 2:
        # Menukar 2 kendaraan (di dalam maupun di luar kapal)
        idx1, idx2 = random.sample(range(n), 2)
        p1 = neighbor.placements[idx1]
        p2 = neighbor.placements[idx2]

        p1["is_loaded"], p2["is_loaded"] = p2["is_loaded"], p1["is_loaded"]
        p1["x"], p2["x"] = p2["x"], p1["x"]
        p1["y"], p2["y"] = p2["y"], p1["y"]

    elif move_type == "rotate":
        # Mengubah orientasi kendaraan
        idx = random.randint(0, n - 1)
        if neighbor.placements[idx]["orientation"] == "Horizontal":
            neighbor.placements[idx]["orientation"] = "Vertical"
        else:
            neighbor.placements[idx]["orientation"] = "Horizontal"
```

```
            # Geser x dan y jika setelah dirotasi koordinatnya keluar batas kapal
            w, l = neighbor.get_vehicle_size(idx)
            max_x = max(0, self.ship.width - w)
            max_y = max(0, self.ship.length - l)
            neighbor.placements[idx]["x"] = min(neighbor.placements[idx]["x"], max_x)
            neighbor.placements[idx]["y"] = min(neighbor.placements[idx]["y"], max_y)

        else:
            # Memindahkan sebuah kendaraan ke suatu koordinat yang berbeda
            idx = random.randint(0, n - 1)
            w, l = neighbor.get_vehicle_size(idx)

            max_x = max(0, self.ship.width - w)
            max_y = max(0, self.ship.length - l)

            old_x = neighbor.placements[idx]["x"]
            old_y = neighbor.placements[idx]["y"]

            # Pastikan koordinat baru berbeda dari koordinat sebelumnya
            if max_x > 0 or max_y > 0:
                new_x = random.randint(0, max_x)
                new_y = random.randint(0, max_y)
                attempts = 0
                while (new_x == old_x and new_y == old_y) and attempts < 20:
                    new_x = random.randint(0, max_x)
                    new_y = random.randint(0, max_y)
                    attempts += 1
                neighbor.placements[idx]["x"] = new_x
                neighbor.placements[idx]["y"] = new_y
            else:
                neighbor.placements[idx]["x"] = 0
                neighbor.placements[idx]["y"] = 0

            # Peluang 25% untuk mengubah status di dalam / di luar kapal
            if random.random() < 0.25:
                neighbor.placements[idx]["is_loaded"] = not neighbor.placements[idx]["is_loaded"]
            else:
                neighbor.placements[idx]["is_loaded"] = True

        return neighbor

    def to_dict(self):
        loaded = []
        unloaded = []
        total_weight = 0

        for i in range(len(self.vehicles)):
            v = self.vehicles[i]
            p = self.placements[i]
            info = {
```

```
            "id": v.id,
            "width": v.width,
            "length": v.length,
            "orientation": p["orientation"],
            "x": p["x"] if p["is_loaded"] else None,
            "y": p["y"] if p["is_loaded"] else None,
            "shipping_fee": v.shipping_fee,
            "weight": v.weight,
            "eta": v.eta
        }
        if p["is_loaded"]:
            loaded.append(info)
            total_weight += v.weight
        else:
            unloaded.append(info)

    return {
        "objective_value": self.get_objective_value(),
        "is_valid": self.is_valid(),
        "total_weight": total_weight,
        "loaded_vehicles": loaded,
        "unloaded_vehicles": unloaded
    }
```

File state.py berlaku sebagai representasi dari satu kandidat state di dalam ruang pencarian. Bagian yang pentingnya tuh fungsi get_objective_value() sebagai fungsi evaluasi untuk memandu saat algoritma berjalan sambil ngitung total keuntungan bersihnya yang disesuaikan dengan kalau ada denda penalti baik itu tumpah tindih, keluar batas, atau melebihi bebas. Selain itu, pergerakan eksplorasi algoritma merujuk pada fungsi get_random_neighbor() yang membangkitkan kandidat state tetangga baru dengan melakukan modifikasi langkah kecil. Contoh memutar posisi V1 dari horizontal ke vertikal, menggeser koordinat di geladak, atau menukar posisi V1 dengan kendaraan lain di luar kapal.

a. Hill-Climbing Steepest Ascent

Nama File	
steepest.py	Implementasi dari algoritma Hill-Climbing Steepest Ascent
Variabel	
initial_state	Solusi awal yang digunakan sebagai *starting point* algoritma

max_iterations	Batas maksimum iterasi pencarian lokal untuk mencegah terjadinya infinite loop
num_neighbors	Jumlah sampel tetangga (neighbor) yang dibangkitkan secara acak pada setiap iterasi
start_time , duration	Menyimpan waktu mulai eksekusi dan total durasi (dalam detik) proses pencarian solusi
current_state	Solusi (state) yang sedang dievaluasi pada iterasi tertentu
current_value	Nilai objective function dari current_state yang sedang dievaluasi
obj_history	List untuk menyimpan riwayat nilai objective function di setiap iterasi untuk keperluan plotting
iterations	Menghitung jumlah iterasi yang berhasil dilakukan oleh algoritma hingga mencapai konvergensi
best_neighbor	Solusi tetangga terbaik yang ditemukan dari seluruh sampel pada satu iterasi berjalan
best_neighbor_value	Nilai objective function terbaik di antara semua tetangga pada iterasi tersebut
Fungsi dan Prosedur	
run_steepest_ascent (initial_state, max_iterations, num_neighbors)	Fungsi utama yang menjalankan algoritma Steepest Ascent dengan mengevaluai sejumlah tetangga secara acak dan memperbarui state jika ditemukan nilai objektive yang lebih tinggi, lalu mengembalikan dictionary berisi hasil akhir

Berikut merupakan *source code* steepest.py

```
import time

def run_steepest_ascent(initial_state, max_iterations=1000, num_neighbors=50):
    start_time = time.time()

    current_state = initial_state
    current_value = current_state.get_objective_value()
```

```
    obj_history = [current_value]
    iterations = 0

    print(f"Mulai Steepest Ascent (Initial Value: {current_value})")

    for i in range(max_iterations):
        iterations += 1
        best_neighbor = None
        best_neighbor_value = -float('inf')

        for _ in range(num_neighbors):
            neighbor = current_state.get_random_neighbor()
            val = neighbor.get_objective_value()

            if val > best_neighbor_value:
                best_neighbor = neighbor
                best_neighbor_value = val

        if best_neighbor_value > current_value:
            current_state = best_neighbor
            current_value = best_neighbor_value
            obj_history.append(current_value)
        else:
            print(f"Local Optimum tercapai pada iterasi {iterations}.")
            break

    duration = time.time() - start_time

    return {
        "initial_value": initial_state.get_objective_value(),
        "best_value": current_value,
        "best_state": current_state,
        "iterations": iterations,
        "duration": duration,
        "obj_history": obj_history
    }
```

Nama File	
main_steepest.py	Skrip runner utama untuk mengeksekusi eksperimen Steepest Ascent, menghasilkan plot, dan menyimpan data hasil ke JSON
Variabel	
output_dir	Direktori tujuan (.../output) untuk menyimpan seluruh berkas hasil eksperimen

	berupa plot visualisasi dan metrik
all_results	List yang menampung rangkuman hasil (run_summary) dari seluruh percobaan (3 runs) untuk diekspor ke JSON
run_num	Penanda nomor iterasi percobaan pada eksperimen (dijalankan dari iterasi 1 hingga 3)
seeds	Parameter opsional berupa list yang berisi daftar seed kustom untuk tiap run (jika diberikan)
current_seed	Nilai acak berdasarkan seed kustom yang diberikan atau berdasarkan waktu (time.time_ns()) agar tiap run memiliki inisialisasi unik
ship, vehicle	Menyimpan instance data kapal dan daftar kendaraan dari default problem yang akan disusun
initial_state	Objek state awal sebelum optimasi dilakukan pada tiap run
result	Dictionary penampung nilai kembalian dari run_steepest_ascent, mencakup nilai akhir, iterasi, durasi, dan best state
final_state	Objek solusi (state) akhir paling optimal yang didapat setelah pencarian local search selesai
states_path, metrics_path	Jalur direktori penyimpanan file gambar untuk plot visualisasi sususnan kendaraan dan grafik evaluasi konvergensi
run_summary	Dictionary yang merangkum data spesifik dari setiap iterasi (nomor run, nilai awal/akhir, iterasi, validitas, durasi) untuk disimpan ke JSON
json_path	Jalur penyimpanan file steepest_results.json yang berisi rekaman data keseluruhan runs dalam format JSON
Fungsi dan Prosedur	

main (seeds=None)	Prosedur yang mengatur perulangan 3 runs eksperimen, menginisialisasi keacakan (bisa menerima input seed kustom atau waktu), memanggil fungsi pembuatan grafik, dan mengekspor objek run_summary ke dalam output/steepst_results.json

Berikut merupakan *source code* main_steepest.py

```
import time

def run_steepest_ascent(initial_state, max_iterations=1000, num_neighbors=50):
    start_time = time.time()

    current_state = initial_state
    current_value = current_state.get_objective_value()

    obj_history = [current_value]
    iterations = 0

    print(f"Mulai Steepest Ascent (Initial Value: {current_value})")

    for i in range(max_iterations):
        iterations += 1
        best_neighbor = None
        best_neighbor_value = -float('inf')

        for _ in range(num_neighbors):
            neighbor = current_state.get_random_neighbor()
            val = neighbor.get_objective_value()

            if val > best_neighbor_value:
                best_neighbor = neighbor
                best_neighbor_value = val

        if best_neighbor_value > current_value:
            current_state = best_neighbor
            current_value = best_neighbor_value
            obj_history.append(current_value)
        else:
            print(f"Local Optimum tercapai pada iterasi {iterations}.")
            break

    duration = time.time() - start_time

    return {
        "initial_value": initial_state.get_objective_value(),
        "best_value": current_value,
        "best_state": current_state,
```

```
        "iterations": iterations,
        "duration": duration,
        "obj_history": obj_history
    }import os
import time
import json
import random
from main import get_default_problem
from state import ShipState
from algo.steepest import run_steepest_ascent
from visualizer import save_state_visualization, save_steepest_plots
def main(seeds=None):
    output_dir = "../output"
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    all_results = []

    print("\n" + "=" * 50)
    print("      EKSPERIMEN STEEPEST ASCENT (3 RUNS)     ")
    print("=" * 50)

    for run_num in range(1, 4):
        if seeds is not None and len(seeds) >= run_num:
            current_seed = seeds[run_num - 1]
        else:
            current_seed = int(time.time_ns() % 1000000)
        random.seed(current_seed)

        ship, vehicles = get_default_problem()
        initial_state = ShipState(ship, vehicles)

        result = run_steepest_ascent(
            initial_state=initial_state,
            max_iterations=500,
            num_neighbors=50
        )

        final_state = result["best_state"]

        print(f"\n--- Run {run_num} ---")
        print(f"Seed Initial    : {current_seed}")
        print(f"Nilai Awal      : {result['initial_value']}")
        print(f"Nilai Akhir     : {result['best_value']}")
        print(f"Total Iterasi   : {result['iterations']}")
        print(f"Waktu Eksekusi  : {round(result['duration'], 4)} detik")
        print(f"Status Valid    : {final_state.is_valid()}")

        states_path = os.path.join(output_dir, f"steepest_states_run_{run_num}.png")
        save_state_visualization(initial_state, final_state, run_num, states_path)
```

```
        metrics_path = os.path.join(output_dir, f"steepest_metrics_run_{run_num}.png")
        save_steepest_plots(result, run_num, metrics_path)

        run_summary = {
            "run_num": run_num,
            "seed": current_seed,
            "initial_value": result["initial_value"],
            "best_value": result["best_value"],
            "iterations": result["iterations"],
            "duration_seconds": round(result["duration"], 4),
            "is_valid": final_state.is_valid(),
            "obj_history": result["obj_history"]
        }
        all_results.append(run_summary)

        time.sleep(0.01)


    json_path = os.path.join(output_dir, "steepest_results.json")
    with open(json_path, "w") as f:
        json.dump(all_results, f, indent=4)

    print("\n" + "=" * 50)
    print(f"Eksperimen Steepest Ascent selesai.")
    print(f"Gambar plot dan file 'steepest_results.json' berhasil tersimpan di folder output!")

if __name__ == "__main__":
    main()
```

Algoritma Steepest Ascent Hill-Climbing pada kasus kapal ini diimplementasiin ke fungsi run_steepest_ascent() pada file steepest.py. Proses pencariannya itu dimulai dengan memuat konfigurasi acak dari initial_state dan langsung mengkalkulasi nilai evaluasi awalnya. Pada setiap iterasi, algoritmanya akan eksplor ruang pencarian dengan dia akan membangkitkan sekumpulan solusi tetangga secara berurutan sesuai parameter num_neighbors (nilai default 50 tetangga). Hal ini mentrigger pemanggilan fungsi yang get_random_neighbor() untuk memodifikasi tata letak kendaraan lewat operasi acar seperti swap, move, atau rotate.

Seluruh kandidat tetangga yang dibangkitkan akan saling dikomparasi untuk mencari satu tetangga dengan nilai objektif mutlak tertinggi (best_neighbor_value). Perpindahan posisi (state) pencarian hanya akan dieksekusi apabila tetangga terbaik tersebut terbukti secara tegas melampaui nilai kondisi saat ini (best_neighbor_value > current_value). Penggunaan syarat perbandingan ini tuh membuat algoritma menolak langkah yang lebih buruk maupun sama (makannya syaratnya kan akan terminate kalau

neighbor.value <= current.value) sesuai di materi modul 03 Beyond Classical Search. Jika tidak ada satupun tetangga yang mampu mengungguli konfigurasi saat ini, pencarian akan dihentikan seketika melalui perintah break dan membuatnya menjadi di titik local optimum (telah tercapai), sekalipun batas maksimal iterasi belum tersentuh.

Untuk mengoordinasikan jalannya eksperimen, file main_steepest.py mengambil peran sebagai skrip prngontrol utama. Skripnya mengeksekusi algoritma di atas sebaanyak 3 kali percobaan berturut-turut (3 runs). Agar variasi tata letak kapal awal (initial_state) selalu objektif, skrip secara otomatis membangkitkan nilai seed acak berbasis waktu setiap kali run baru dimulai. Setelah tiap siklus percobaan selesai, fungsi ini memanggil modul visualisasi buat merender ilustrasi penempatan kendaraan (save_state_visualization) serta mem-plot grafik konvergensi (save_steepest_plots). Terakhir, seluruh atribut teknis percobaan beserta riwayat iterasinya (obj_history) diekspor secara utuh ke dalam file luaran steepest_results.json sebagai landasan data analsisinya.

b. Simulated Annealing

Nama File	
sim_annealing,py	Implementasi dari algoritma Simulated Annealing
Variabel	
initial_state	State awal (ShipState) yang digunakan sebagai titik mulai pencarian
initial_temp	Suhu awal algoritma (default 1000.0)
cooling_rate	Faktor pendinginan geometris, suhu dikalikan nilai ini di setiap iterasi (default 0.995)
min_temp	Suhu minimum, pencarian berhenti jika suhu sudah mencapai nilai ini (default 0.01)
max_iterations	Batas maksimum iterasi pencarian (default 2000)
stuck_limit	Jumlah iterasi berturut-turut tanpa perubahan nilai current agar agen dianggap stuck (default 50)
start_time, duration	Menyimpan waktu mulai eksekusi dan total

	durasi (dalam detik) proses pencarian
current_state, current_val	Solusi (state) yang sedang ditempati pada iterasi tertentu dan nilai objective function-nya
best_state, best_val	Solusi terbaik yang pernah ditemukan selama pencarian beserta nilai objective function-nya (elitism)
neighbor, neighbor_val	Solusi tetangga acak yang dibangkitkan pada iterasi berjalan dan nilai objective function-nya
delta_e	Selisih nilai objektif (neighbor_val - current_val), bernilai positif jika tetangga lebih baik karena SA ini memaksimalkan objective function
prob	Probabilitas menerima langkah yang lebih buruk, dihitung dengan rumus e^(delta_e / temp)
accept	Penanda boolean apakah tetangga diterima atau tidak pada iterasi berjalan
temp	Suhu pada iterasi berjalan
iteration	Menghitung jumlah iterasi yang telah dilakukan oleh algoritma
no_change_counter	Penghitung iterasi berturut-turut di mana nilai current tidak berubah (langkah ditolak atau sideways move)
stuck_frequency	Jumlah episode stuck, bertambah satu setiap kali no_change_counter mencapai stuck_limit
stuck_iterations	List nomor iterasi saat setiap episode stuck terdeteksi
obj_history, best_obj_history	List riwayat nilai objective function current dan nilai terbaik di setiap iterasi untuk keperluan plotting
prob_iterations, prob_history	List nomor iterasi dan nilai probabilitas e^(delta_e / temp) pada setiap langkah buruk yang dievaluasi, untuk plot probabilitas

	penerimaan
Fungsi dan Prosedur	
run_simulated_annealing (initial_state, initial_temp, cooling_rate, min_temp, max_iterations, stuck_limit)	Fungsi utama untuk menjalankan algoritma Simulated Annealing. Pada setiap iterasi dibangkitkan satu tetangga acak, tetangga diterima jika lebih baik atau sama (delta_e >= 0), atau dengan probabilitas e^(delta_e / temp) jika lebih buruk. Suhu diturunkan hingga mencapai min_temp atau max_iterations. Akan mengembalikan state awal, akhir, dan terbaik, nilai objektif, durasi, jumlah iterasi, frekuensi stuck, serta riwayat untuk plotting.
copy_state() (di dalam state.py)	Method ShipState untuk menyalin state secara independen (deep copy placements)
get_objective_value() (di dalam state.py)	Menghitung objective function → total shipping fee kendaraan yang dimuat dikurangi penalti (500 x sel yang overlap + 500 x sel di luar batas + 200 x kelebihan berat)
get_random_neighbor() (di dalam state.py)	Memilih satu tetangga acak melalui salah satu operasi dari swap (menukar dua kendaraan), move (memindahkan kendaraan ke koordinat baru), atau rotate (mengubah orientasi)

Berikut merupakan *source code* sim_annealing.py

```
import math
import random
import time


def run_simulated_annealing(initial_state, initial_temp=1000.0, cooling_rate=0.995,
min_temp=0.01, max_iterations=2000, stuck_limit=50):
    start_time = time.time()

    current_state = initial_state.copy_state()
    current_val = current_state.get_objective_value()

    best_state = current_state.copy_state()
    best_val = current_val
```

```
temp = initial_temp
iteration = 0

# Stuck = nilai current ga berubah2 selama stuck_limit iterasi berturut-turut,
# artinya agen itu stuck di local maximum / flat local maximum (plateau) / shoulder.
# Sideways move (delta_e == 0) ga diitung sebagai kemajuan juga.
no_change_counter = 0
stuck_frequency = 0
stuck_iterations = []

obj_history = [current_val]
best_obj_history = [best_val]
prob_iterations = []
prob_history = []

while temp > min_temp and iteration < max_iterations:
    iteration += 1

    neighbor = current_state.get_random_neighbor()
    neighbor_val = neighbor.get_objective_value()

    delta_e = neighbor_val - current_val

    if delta_e > 0:
        accept = True
    elif delta_e == 0:
        accept = True
    else:
        prob = math.exp(delta_e / temp)
        prob_iterations.append(iteration)
        prob_history.append(prob)

        if random.random() < prob:
            accept = True
        else:
            accept = False

    if accept:
        current_state = neighbor
        current_val = neighbor_val

        if current_val > best_val:
            best_state = current_state.copy_state()
            best_val = current_val

    # nilai current berubah (naik, atau turun yg diterima) -> agen tidak stuck
    if accept and delta_e != 0:
        no_change_counter = 0
    else:
        # ditolak atau sideways -> nilai current tetap
```

```
            no_change_counter += 1
            # hitung 1x per episode stuck (saat pertama kali mencapai batas)
            if no_change_counter == stuck_limit:
                stuck_frequency += 1
                stuck_iterations.append(iteration)

        obj_history.append(current_val)
        best_obj_history.append(best_val)

        # turunkan suhu
        temp = temp * cooling_rate

    duration = time.time() - start_time

    return {
        "initial_state": initial_state,
        "final_state": current_state,
        "best_state": best_state,
        "initial_value": initial_state.get_objective_value(),
        "final_value": current_val,
        "best_value": best_val,
        "duration": duration,
        "iterations": iteration,
        "stuck_frequency": stuck_frequency,
        "stuck_iterations": stuck_iterations,
        "obj_history": obj_history,
        "best_obj_history": best_obj_history,
        "prob_iterations": prob_iterations,
        "prob_history": prob_history
    }
```

Nama File	
main_sa.py	
Variabel	
ship	Objek Ship hasil get_default_problem() yang berisi dimensi geladak kapal (lebar 10, panjang 16) dan kapasitas berat maksimum 50
vehicles	List 10 objek Vehicle (V1-V10) hasil get_default_problem() yang berisi dimensi, tarif pengiriman, berat, dan ETA
output_dir	Direktori tujuan (../output) untuk menyimpan

	plot dan file JSON hasil eksperimen
Fungsi dan Prosedur	
main()	Prosedur yang mengambil problem default melalui get_default_problem(), lalu menjalankan eksperimen Simulated Annealing dengan memanggil run_simulated_annealing_experiment(ship, vehicles, output_dir)

Berikut merupakan *source code* main_sa.py

```
from main import get_default_problem, run_simulated_annealing_experiment


def main():
    ship, vehicles = get_default_problem()
    run_simulated_annealing_experiment(ship, vehicles, output_dir="../output")


if __name__ == "__main__":
    main()
```

Algoritma Simulated Annealing pada penataan kendaraan di kapal bekerja dengan menelusuri ruang konfigurasi ShipState secara acak sambil mengizinkan perpindahan ke konfigurasi yang lebih buruk dengan peluang yang mengecil seiring turunnya suhu. Implementasinya dibagi menjadi sim_annealing.py (algoritma), main_sa.py (pemanggil, shortcut), dan run_simulated_annealing_experiment() pada main.py (pengatur algoritma).

Pencarian dimulai dari initial_state berupa penempatan acak (status muat, koordinat, dan orientasi tiap kendaraan) dengan suhu awal 1000. Pada setiap iterasi, get_random_neighbor() membangkitkan satu tetangga melalui salah satu operasi swap, move, atau rotate. Selisih nilai objektif dihitung sebagai ΔE = neighbor_val-current_val. Karena tujuannya memaksimalkan keuntungan, tetangga langsung diterima jika $\Delta E > 0$. Jika $\Delta E = 0$ (sideways move), tetangga juga diterima. Jika $\Delta E < 0$, tetangga diterima dengan probabilitas e^(ΔE/T). Mekanisme ini memungkinkan algoritma keluar dari local maximum, terutama pada suhu tinggi ketika pemindahan ke state yang lebih buruk masih diterima. Setelah itu suhu diturunkan secara geometris ($T \leftarrow T \times 0.995$) sehingga pencarian bergeser dari eksplorasi ke eksploitasi.

Selama proses, best_state selalu diperbarui ketika ditemukan nilai yang lebih tinggi, sehingga solusi terbaik tidak hilang walaupun langkah buruk diterima. Algoritma juga menghitung frekuensi stuck, yaitu berapa kali nilai current tidak berubah (bisa dari

ditolak atau sideways) selama 50 iterasi berturut-turut, yang menandakan terjebak di local maximum, plateau, atau shoulder. Pencarian berhenti ketika suhu mencapai 0.01 atau iterasi mencapai maksimum yaitu 2000.

Pada main.py, eksperimen dijalankan tiga kali dengan seed berbeda (kami telah menetapkan benchmark 42, 100, dan 2026 agar initial state sama dengan algoritma lain dan dapat dibandingkan). Hasil tiap run disimpan sebagai grafik, visualisasi state awal dan akhir, serta ringkasan sa_results.json.

c. Genetic Algorithm

Nama File	
genetic_algorithm.py	Implementasi dari algoritma Genetic Algorithm
Variabel	
initial_population	Kumpulan populasi awal berisi sejumlah k individu (ShipState) acak.
population_size	Jumlah individu dalam satu generasi (ukuran populasi k).
max_iterations	Batas maksimum iterasi (banyaknya generasi yang dibangkitkan).
mutation_rate	Peluang terjadinya mutasi pada individu anak (0.1 atau 10%).
fitness_values	Nilai fitness terukur dari setiap individu untuk seleksi.
initial_state	Individu terbaik yang ditemukan pada populasi generasi awal.
final_state	Individu terbaik pada populasi generasi terakhir.
best_state	Individu terbaik yang pernah ditemukan selama keseluruhan proses evolusi.
max_obj_history	Riwayat nilai fungsi objektif maksimum populasi di setiap generasi.

avg_obj_history	Riwayat nilai fungsi objektif rata-rata populasi di setiap generasi.
Fungsi dan Prosedur	
get_fitness_values(population)	Menghitung skor fitness dari nilai fungsi objektif. Jika terdapat nilai objektif bernilai negatif akibat penalti pelanggaran, diterapkan pergeseran skala linier (fitness scaling): $fitness = val - min_v al + 1$, sehingga seluruh individu memiliki fitness ≥ 1 dan gradien kualitas antar-individu tetap terjaga.
random_selection(pop, fitness)	Memilih satu individu induk menggunakan metode Roulette Wheel Selection secara proporsional terhadap nilai fitness-nya ($P_i = \frac{f_i}{\Sigma f}$).
reproduce(x, y)	Menggabungkan dua induk x dan y menggunakan Single-Point Crossover pada indeks pemotongan acak $c \in [1, N-1]$ untuk menghasilkan satu individu anak
mutate(child)	Menerapkan mutasi pada anak melalui pemanggilan child.get_random_neighbor().
get_best_individual(pop)	Mengembalikan individu dengan nilai fungsi objektif tertinggi dalam populasi.
run_genetic_algorithm(...)	Fungsi utama pengendali alur siklus evolusi, pencatatan histori konvergensi, dan perekaman transisi untuk visualisasi animasi.

Berikut merupakan *source code* implementasi Genetic Algorithm:

```
import random
import time
```

```
from state import ShipState
def get_fitness_values(population):
    obj_values = [ind.get_objective_value() for ind in population]
    min_val = min(obj_values)
    if min_val < 0:
        return [val - min_val + 1 for val in obj_values]
    else:
        return [val + 1 for val in obj_values]
def random_selection(population, fitness_values):
    total = sum(fitness_values)
    pick = random.uniform(0, total)
    current = 0
    for i in range(len(population)):
        current += fitness_values[i]
        if current >= pick:
            return population[i]
    return population[-1]
def reproduce(x, y):
    n = len(x.placements)
    if n < 2:
        return x.copy_state()
    c = random.randint(1, n - 1)
    child_placements = []
    for p in x.placements[:c]:
        child_placements.append(dict(p))
    for p in y.placements[c:]:
        child_placements.append(dict(p))
    return ShipState(x.ship, x.vehicles, child_placements)
def mutate(child):
    return child.get_random_neighbor()
def get_best_individual(population):
    best_ind = population[0]
    best_val = best_ind.get_objective_value()
    for i in range(1, len(population)):
        curr_ind = population[i]
        curr_val = curr_ind.get_objective_value()
        if curr_val > best_val:
            best_ind = curr_ind
            best_val = curr_val
    return best_ind
def run_genetic_algorithm(initial_population, max_iterations=150, mutation_rate=0.1,
record_history=False):
    start_time = time.time()
    population = [ind.copy_state() for ind in initial_population]
    population_size = len(population)
    initial_state = get_best_individual(population).copy_state()
    best_state = initial_state.copy_state()
    best_val = best_state.get_objective_value()
    max_obj_history = []
    avg_obj_history = []
```

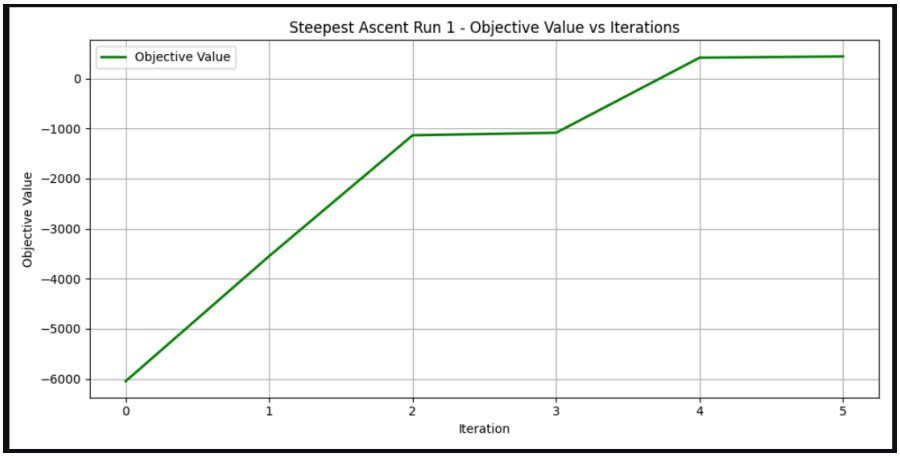
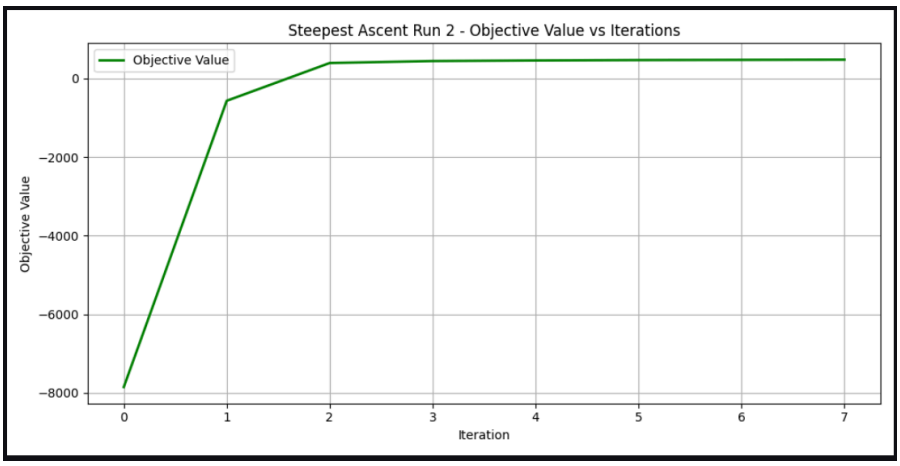
```
    history = []
    if record_history:
        history.append({"iteration": 0, "state": initial_state.copy_state()})
    obj_values = [ind.get_objective_value() for ind in population]
    max_obj_history.append(max(obj_values))
    avg_obj_history.append(sum(obj_values) / len(obj_values))
    sample_step = max(1, max_iterations // 40)
    for iteration in range(1, max_iterations + 1):
        fitness_values = get_fitness_values(population)
        new_population = []
        for i in range(population_size):
            x = random_selection(population, fitness_values)
            y = random_selection(population, fitness_values)
            child = reproduce(x, y)
            if random.random() < mutation_rate:
                child = mutate(child)
            new_population.append(child)
        population = new_population
        obj_values = [ind.get_objective_value() for ind in population]
        curr_max = max(obj_values)
        max_obj_history.append(curr_max)
        avg_obj_history.append(sum(obj_values) / len(obj_values))
        found_better = False
        if curr_max > best_val:
            current_best = get_best_individual(population)
            best_state = current_best.copy_state()
            best_val = best_state.get_objective_value()
            found_better = True
        if record_history and (found_better or iteration % sample_step == 0 or iteration ==
max_iterations):
            history.append({"iteration": iteration, "state": best_state.copy_state()})
    final_state = get_best_individual(population).copy_state()
    duration = time.time() - start_time
    return {
        "initial_state": initial_state,
        "final_state": final_state,
        "best_state": best_state,
        "initial_value": initial_state.get_objective_value(),
        "final_value": final_state.get_objective_value(),
        "best_value": best_state.get_objective_value(),
        "population_size": population_size,
        "iterations": max_iterations,
        "mutation_rate": mutation_rate,
        "duration": duration,
        "max_obj_history": max_obj_history,
        "avg_obj_history": avg_obj_history,
        "history": history
    }
```

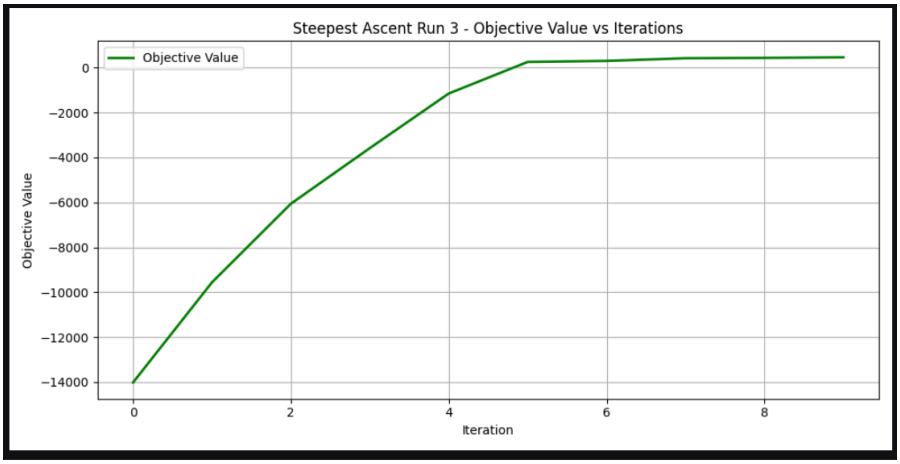
Algoritma *Genetic Algorithm* bekerja pada sekumpulan kandidat solusi atau populasi individu yang berukuran k. Di sini, satu individu merepresentasikan satu ShipState dengan daftar *placements* sepanjang $N = 10$ kendaraan yang berperan sebagai kromosom. Proses pencarian ini dilakukan melalui beberapa tahapan pada setiap generasi, diawali dengan evaluasi *fitness* di mana setiap individu dihitung nilai fungsi objektifnya. Jika terdapat individu yang melanggar batasan fisik sehingga bernilai negatif, maka dilakukan pergeseran linier $val - min_val + 1$ agar seleksi roda rolet (*Roulette Wheel Selection*) dapat beroperasi pada domain positif tanpa kehilangan gradien penalti. Melalui seleksi roda rolet ini, individu dengan nilai *fitness* yang lebih tinggi akan memiliki proporsi peluang yang lebih besar untuk terpilih sebagai induk x dan y. Selanjutnya, tahapan *crossover* dilakukan menggunakan metode *Single-Point Crossover*, di mana titik pemotongan c dipilih secara acak untuk mewariskan gen kendaraan dari indeks 0 s.d. $c - 1$ dari induk x dan gen dari indeks c s.d. $N - 1$ dari induk y. Setelah itu, dengan probabilitas mutasi sebesar mutation_rate = 0.1, individu anak dapat mengalami mutasi melalui pemanggilan fungsi child.get_random_neighbor() yang berupa operasi *swap*, *rotate*, atau *move*. Siklus ini diakhiri dengan *generational replacement*, di mana populasi lama digantikan secara menyeluruh oleh populasi anak (population = new_population), sementara algoritma terus mencatat nilai maksimum dan rata-rata dari setiap generasi hingga batas max_iterations tercapai.

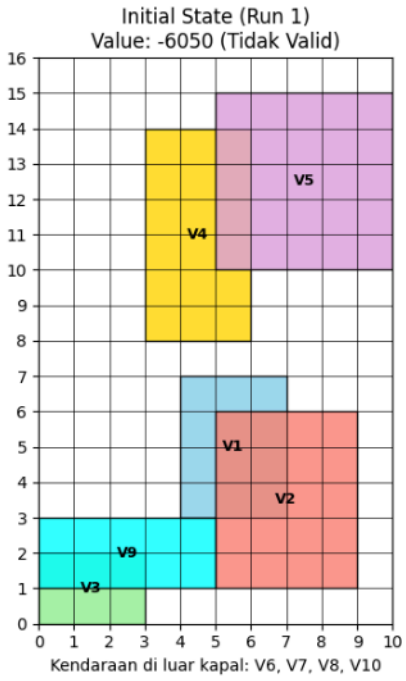
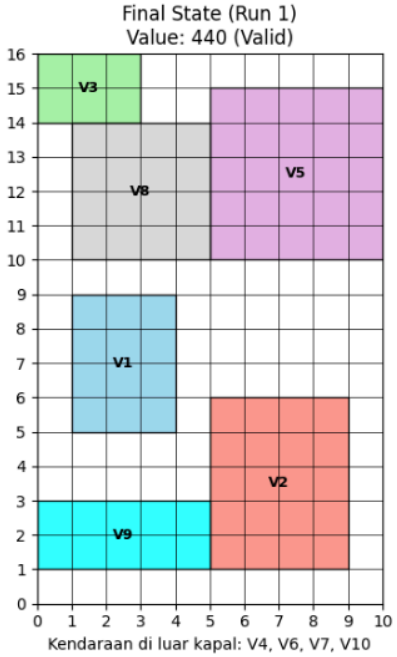
BAB 3
HASIL DAN ANALISIS

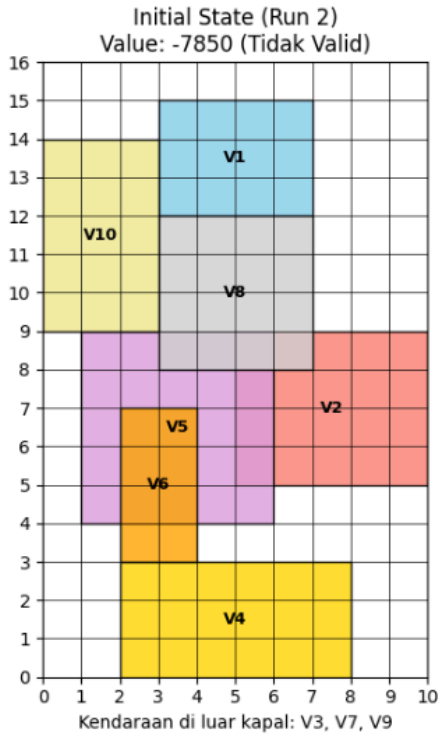
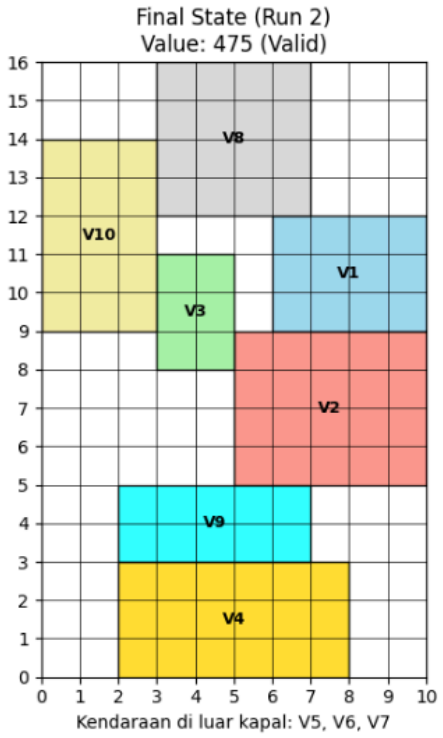
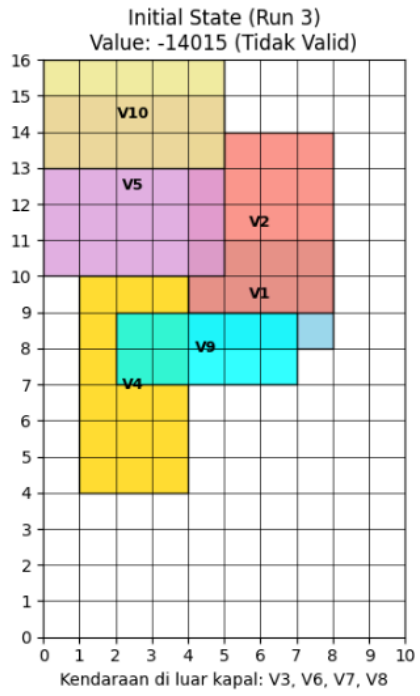
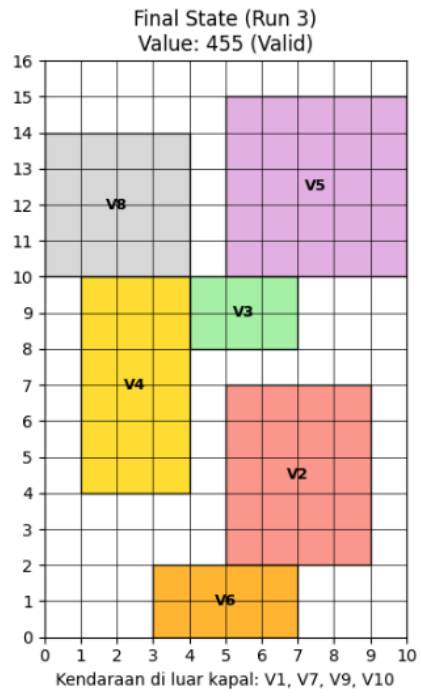
a. Steepest Ascent Hill-Climbing

Diterapkan tiga percobaan yang masing-masing menggunakan seed yang berbeda (42, 100, dan 2026) untuk menghasilkan initial state yang bervariasi pada algoritma Steepest Ascent Hill-Climbing..

Run	Grafik Nilai Objective Function terhadap Banyak Iterasi
1	
2	

3	

Run	Initial State	Final State
1		

2		
3		

Parameter	Run 1	Run 2	Run 3
Durasi (s)	0.025	0.0246	0.035

Total Iterasi	6	8	10
Initial State	-6050 (Tidak Valid)	-7850 (Tidak Valid)	-14015 (Tidak Valid)
Final State	440 (Valid)	475 (Valid)	455 (Valid)

Algoritma *steepest ascent hill-climbing* menunjukkan peningkatan nilai yang sangat drastis pada iterasi-iterasi awal karena secara agresif menghilangkan penalti tabrakan (overlap) antar kendaraan. Sebagai contoh pada Run 1, nilai awal bermula di angka -6050 akibat banyaknya tumpah tindih. Hanya dalam satu kali iterasi pencarian, algoritma memilih langkah pergerakan terbaik (steepest) sehingga nilainya melonjak naik sebesar 2500 poin menjadi -3550. Nilai ini terus menanjak tajam hingga menembus angka positif 415 pada iterasi ke-4 yang menandakan sebagian besar tabrakan telah teratasi. Secara visual, hal ini dibuktikan dengan final state di mana kendaraan di dalam area kapal telah tersusun rapi tanpa tumpang tindih (solusi Valid). Memasuki iterasi akhir, peningkatan nilai objektif mulai mengecil akhirnya grafik mendatar.

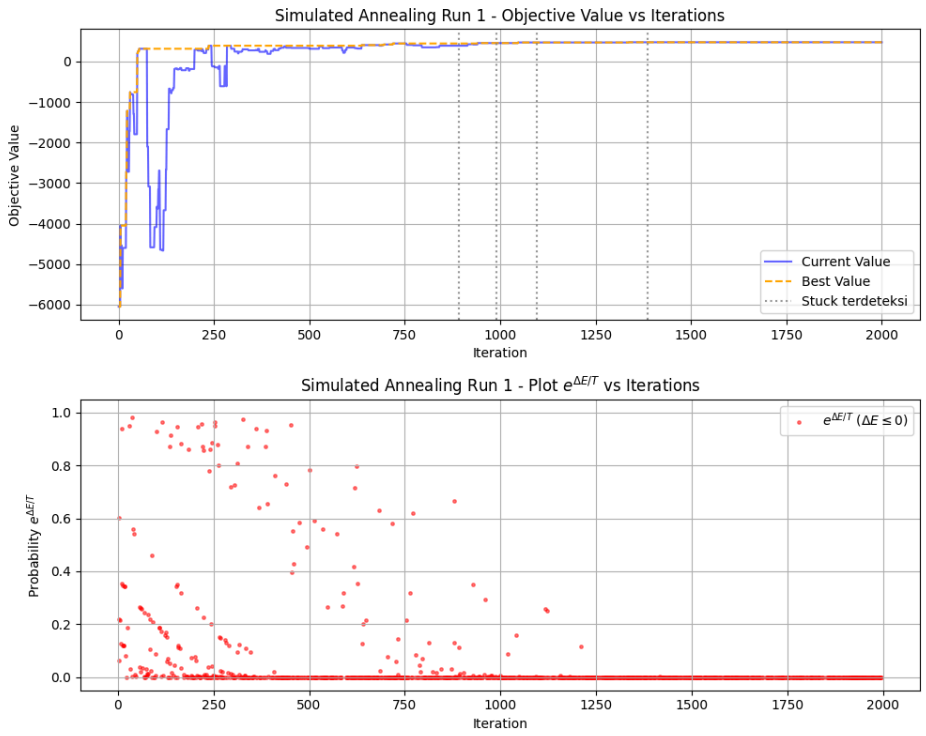
Pada Run 2, lonjakan nilai perlahan menyusut, di mana penginkatan dari iterasi ke-6 ke iterasi ke-7 hanya bernilai 5 poin (bergeser dari 470 menjadi 475). Setelah mencapai nilai 475, grafik terhenti secara total. Hal ini terjadi karena dari seluruh kemungkinan pergeseran letak kendaraan (sebagai neighbors) yang dievaluasi dari titik tersebut, tidak ada satupun yang menghasilkan nilai lebih dari 475. Kondisi inilah yang membuktikan bahwa algoritma telah terjebak di titik local optimum.

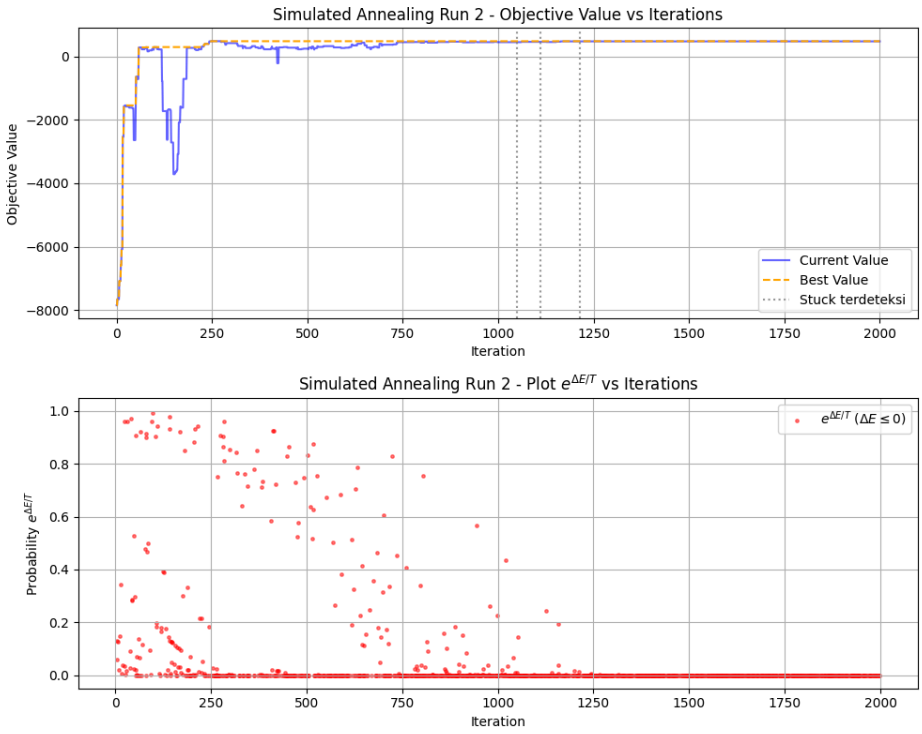
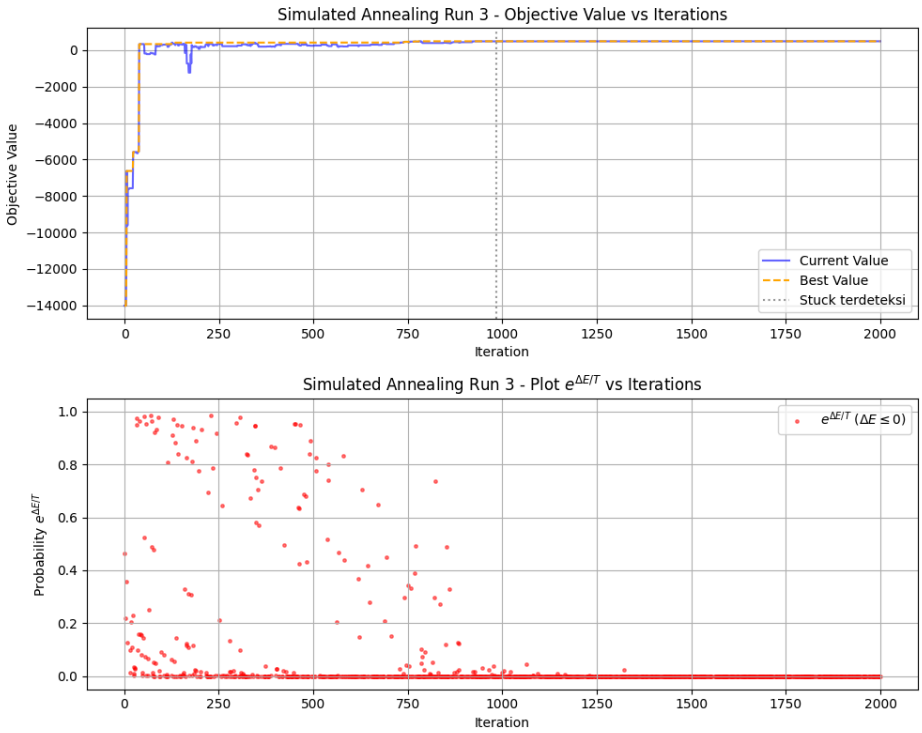
Karena algoritma ini bersifat deterministik dengan selalu mengambil jalur perbaikan yang paling besar di setiap langkah, hasil akhir sangat bergantung pada posisi initial state. Ketiga percobaan membuktikan hal ini secara numerik. Titik awal -6050 (Run 1) berhenti di lokal optimum bernilai 440, titik awal -7850 (Run 2) berhenti di 475, dan titik awal terburuk -14015 (Run 3) berhenti di 455. Menariknya, Run 2 yang memiliki initial state lebih buruk daripada Run 1 justru malah berhasil membuahkan susunan paling optimal secara keseluruhan (nilai tertinggi 475). Fakta ini menegaskan bahwa awalan yang tampak lebih baik tidak menjamin penemuan puncak lokal yang lebih tinggi pada hill-climbing.

b. Simulated Annealing

Dilakukan tiga percobaan untuk algoritma Simulated Annealing dengan seed 42, 100, dan 2026, sehingga initial state yang digunakan identik dengan percobaan Steepest Ascent Hill-Climbing (nilai awal -6.050, -7.850, dan -14.015). Parameter yang digunakan adalah suhu awal 1000, cooling rate 0,995, suhu minimum 0,01, batas iterasi maksimum 2.000, dan stuck_limit 50. Dengan parameter tersebut suhu pada iterasi ke-2.000 baru

mencapai sekitar 0.044, sehingga pencarian berhenti karena mencapai batas iterasi, bukan karena suhu minimum.

Percobaan	Grafik
1	

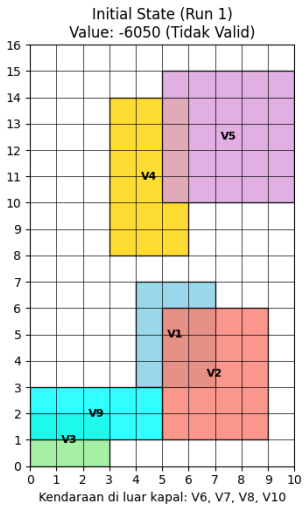
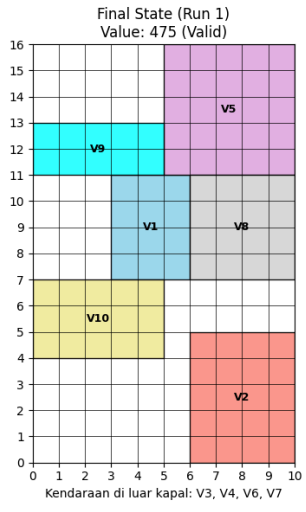
2	
3	

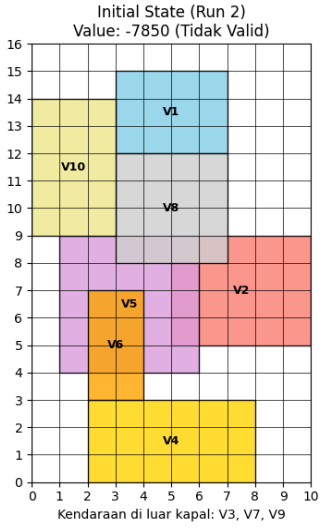
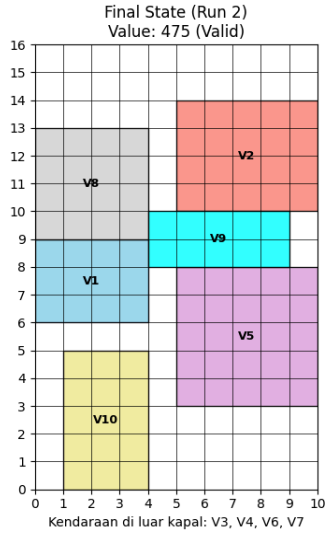
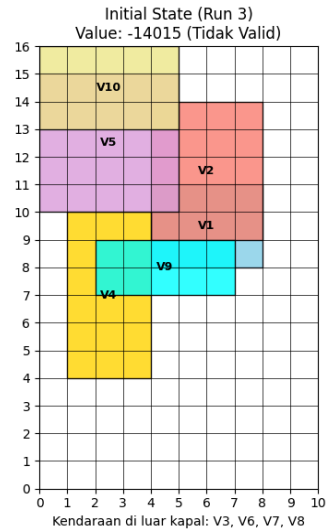
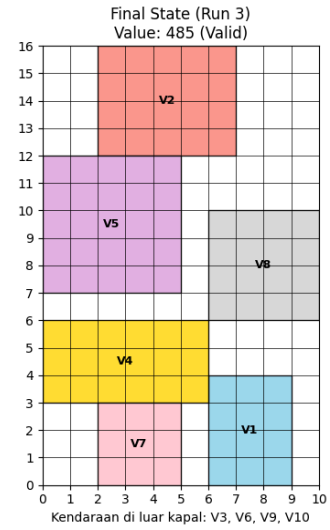
Pada grafik nilai objective function terhadap iterasi, ketiga run menunjukkan pola yang serupa. Dalam sekitar 50 iterasi pertama, nilai current melonjak dari kisaran -6.000 hingga -14.000 menuju sekitar nol atau positif. Langkah yang memperbaiki nilai ($\Delta E > 0$)

selalu diterima, dan langkah awal banyak menghilangkan tabrakan antar kendaraan yang dikenai penalti 500 per sel. Setelah itu, Run 1 dan Run 2 sempat turun kembali tajam, masing-masing ke sekitar -4.500 (iterasi sekitar 70-130) dan sekitar -3.700 (iterasi sekitar 150). Penurunan ini merupakan langkah buruk yang diterima saat suhu masih tinggi ($T \approx 606$ pada iterasi ke-100), dan menjadi ciri khas SA yang tidak dimiliki Hill-Climbing karena Hill-Climbing tidak pernah menerima penurunan nilai. Pada Run 3 penurunan jauh lebih kecil (sekitar -1.200 pada iterasi sekitar 170). Setelah iterasi ke-250 nilai current kembali positif, dan pada sekitar iterasi ke-750 hingga ke-1.000 ($T < 7$) kurva mendatar di 475 untuk Run 1 dan Run 2, serta 485 untuk Run 3.

Grafik e^(ΔE/T) memperlihatkan bagaimana suhu mengendalikan penerimaan langkah buruk. Pada sekitar 500 iterasi pertama ($T > 80$) cukup banyak langkah buruk memiliki probabilitas 0.6-1.0, terutama langkah dengan penurunan kecil seperti mengeluarkan kendaraan yang tarifnya hanya puluhan, sehingga langkah tersebut sering diterima. Seiring turunnya suhu, titik-titik probabilitas bergeser ke bawah. Setelah sekitar iterasi ke-1.250 ($T \approx 2$) hampir seluruh langkah buruk memiliki probabilitas mendekati nol dan ditolak. Banyaknya titik di sekitar nol sepanjang iterasi juga disebabkan oleh besarnya penalti (500 per sel overlap atau di luar batas, 200 per unit kelebihan berat). Langkah yang melanggar batasan menghasilkan |ΔE| ratusan hingga ribuan, yang setelah beberapa ratus iterasi pertama jauh melampaui suhu sehingga hampir selalu ditolak. Dengan demikian pencarian bergeser dari fase eksplorasi ke fase eksploitasi.

Berikut merupakan *state* awal dan *state* akhir dari masing-masing percobaan.

Percobaan	Initial State	Final State
1		

2		
3		

Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.0965	0.0942	0.0962
Frekuensi Stuck	4	3	1
Nilai Objective Function Awal	-6.050	-7.850	-14.015
Nilai Objective Function Akhir	475	475	485

Seluruh run menghasilkan state akhir yang valid. Run 1 dan Run 2 berakhir pada nilai 475 dengan kendaraan V1, V2, V5, V8, V9, dan V10 yang dimuat (total berat 49 dari 50). Run 3 mencapai nilai 485 dengan kendaraan V1, V2, V4, V5, V7, dan V8, yang memenuhi kapasitas beban penuh 50/50 dan merupakan batas optimal pada kasus uji ini. Pada Run 2, best state bernilai 480 (V1, V2, V4, V5, V7, V10), lebih tinggi daripada nilai akhirnya (475). Hal ini menunjukkan bahwa penyimpanan best_state (elitism) berguna, karena langkah buruk yang diterima dapat menjauhkan current state dari solusi terbaik yang pernah ditemukan.

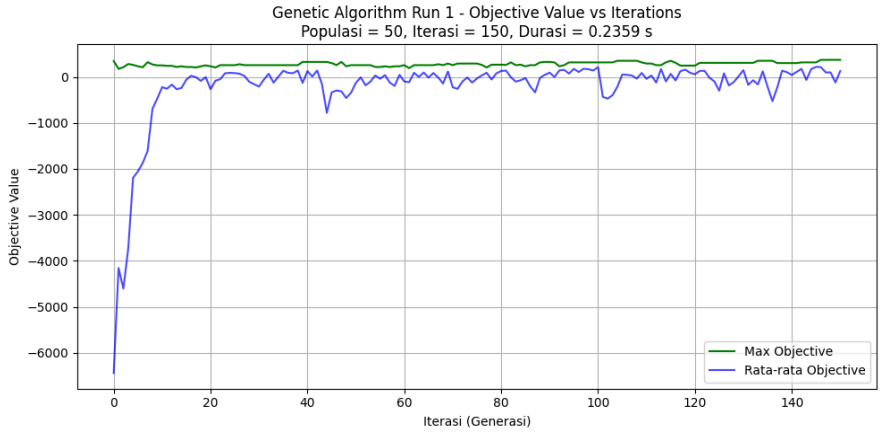
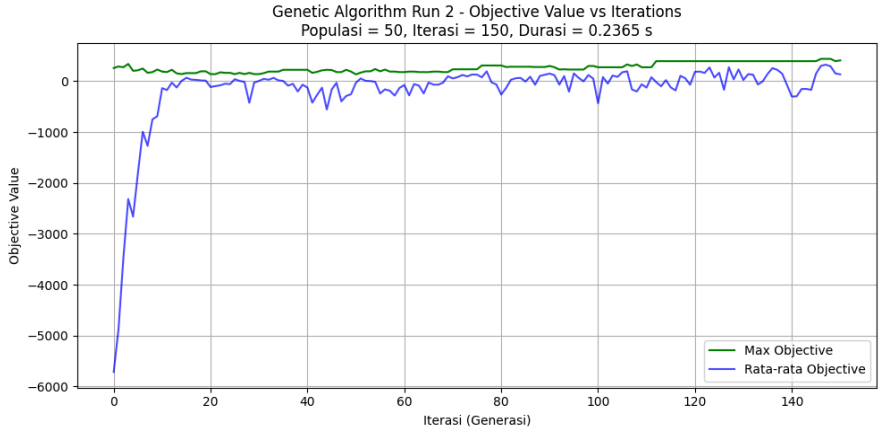
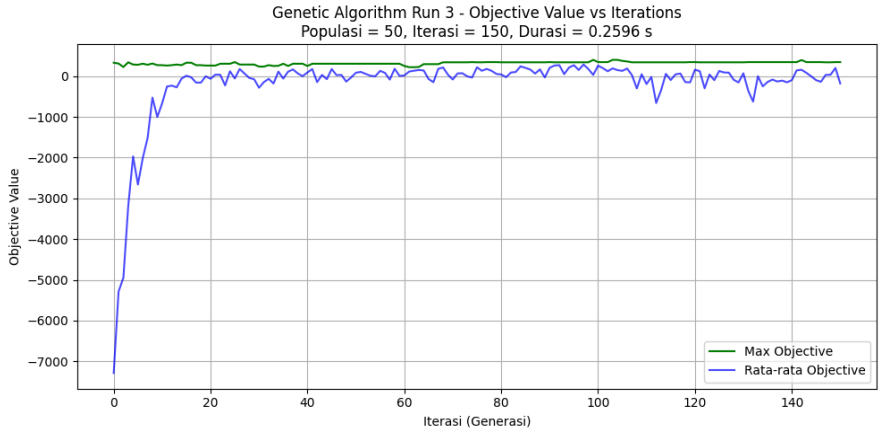
Jika dibandingkan dengan Steepest Ascent pada initial state yang sama, SA menghasilkan nilai akhir yang sama atau lebih baik: 475 berbanding 440 (Run 1), 475 berbanding 475 (Run 2), dan 485 berbanding 455 (Run 3). Run 3 menarik karena memiliki initial state terburuk (-14.015) tetapi justru menghasilkan solusi terbaik, yang mengindikasikan bahwa SA tidak sebergantung Hill-Climbing pada kualitas titik awal. Meski begitu, hanya satu dari tiga run yang mencapai 485, sehingga SA tidak menjamin optimum global dan hasilnya tetap bersifat probabilistik.

Durasi eksekusi ketiga run hampir identik (0,0942-0,0965 detik) karena jumlah iterasi tetap 2.000 dengan satu evaluasi tetangga per iterasi. Hal ini berbeda dengan Steepest Ascent yang durasinya bergantung pada kapan local optimum tercapai. Frekuensi stuck (4, 3, dan 1) menghitung jumlah episode ketika nilai current tidak berubah selama 50 iterasi berturut-turut. Pada Run 1, stuck terdeteksi empat kali (sekitar iterasi 890-1.390), yang berarti pencarian sempat keluar dari plateau lewat langkah yang diterima, namun tetap berakhir di 475. Run 3 hanya mengalami satu episode stuck (sekitar iterasi 980), setelah itu bertahan di 485 hingga iterasi terakhir. Jadi frekuensi stuck yang kecil di sini justru menandakan pencarian sudah ditemukan lebih awal di solusi terbaik, bukan kinerja yang lebih buruk.

c. Genetic Algorithm

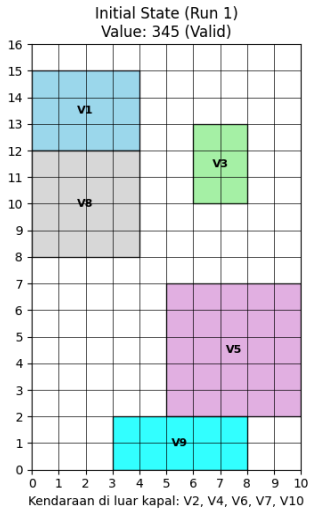
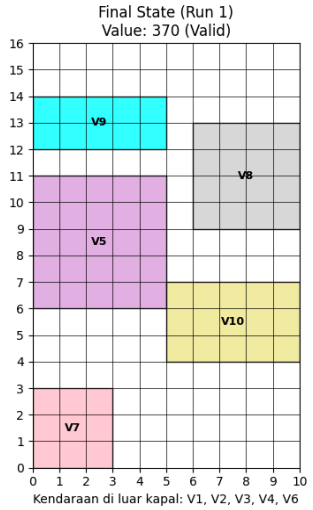
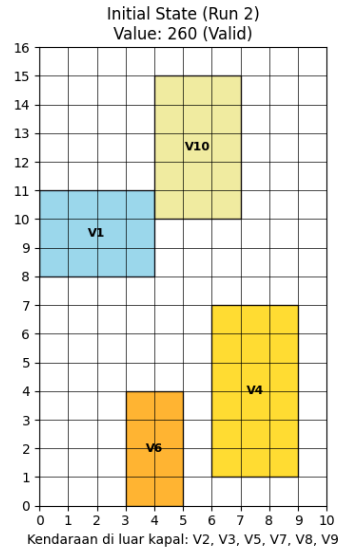
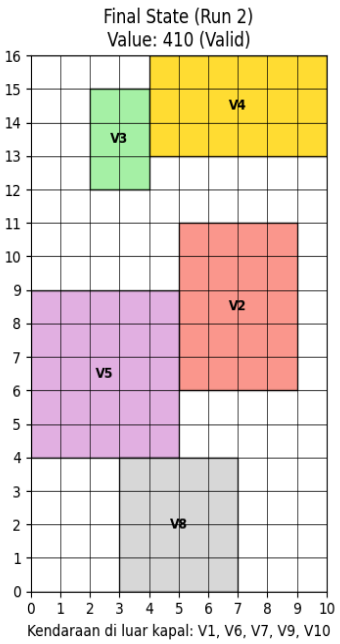
Eksperimen Genetic Algorithm dievaluasi dalam dua skenario pengujian dengan nilai mutation_rate = 0.1. Untuk perwakilan, kami menggunakan populasi 50 dan iterasi 150 dengan seed awal (42, 100, 2026) agar dapat dibandingkan secara adil terhadap Hill-Climbing dan Simulated Annealing. Namun, kode tetap bisa melakukan total 18 runs untuk menunjukkan seluruh pekerjaan Genetic Algorithm dengan memvariasikan jumlah populasi (kontrol iterasi = 150, variasi populasi = 20, 50, 100) dan memvariasikan jumlah iterasi (kontrol populasi = 50, variasi iterasi = 50, 150, 300).

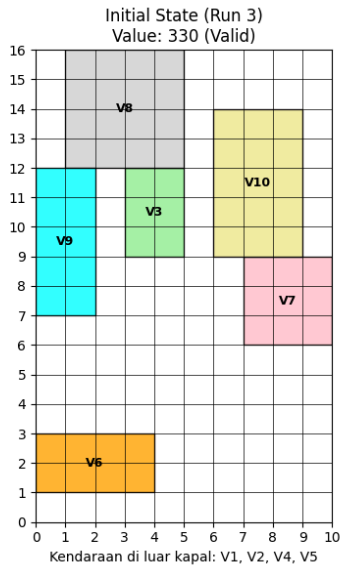
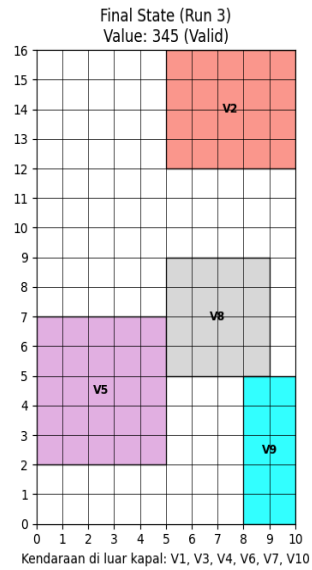
Percobaan	Grafik Average dan Best Objective Function Value Terhadap Jumlah Iterasi

1	
2	
3	

Berikut merupakan *state* awal dan *state* akhir dari masing-masing percobaan.

Percobaan	Initial State	Final State

1	 Initial State (Run 1) Value: 345 (Valid) V1 V8 V3 V5 V9 Kendaraan di luar kapal: V2, V4, V6, V7, V10	 Final State (Run 1) Value: 370 (Valid) V9 V8 V5 V10 V7 Kendaraan di luar kapal: V1, V2, V3, V4, V6
2	 Initial State (Run 2) Value: 260 (Valid) V10 V1 V4 V6 Kendaraan di luar kapal: V2, V3, V5, V7, V8, V9	 Final State (Run 2) Value: 410 (Valid) V4 V3 V2 V5 V8 Kendaraan di luar kapal: V1, V6, V7, V9, V10

3		

Percobaan 3 Run menggunakan probabilitas mutasi 0.1.

Parameter	Percobaan 1	Percobaan 2	Percobaan 3
Durasi (s)	0.2359	0.2365	0.2596
Banyak Populasi	50	50	50
Jumlah Iterasi	150	150	150
Initial State	345	260	330
Final State	370	410	345

Pada grafik Objective Value vs Iterations, terlihat bahwa pada generasi ke-0 nilai avg_obj_history bernilai sangat rendah dan negatif, berada di kisaran -5.500 hingga -7.200. Hal ini disebabkan oleh inisialisasi acak, di mana sebagian besar individu melanggar batasan fisik kapal sehingga terkena penalti besar.

Meskipun rata-rata awal sangat buruk, keberadaan ukuran populasi yang memadai $k \geq 20$ memberikan keanekaragaman genetik yang cukup sehingga hampir selalu terdapat minimal satu individu acak yang valid sejak awal, menghasilkan nilai maksimum awal (initial_value) positif di kisaran 260 hingga 345.

Dalam 10 hingga 20 generasi pertama, seleksi Roulette Wheel Selection yang dibantu oleh fungsi fitness scaling bisa kromosom-kromosom yang bertabrakan. Kurva rata-rata populasi melonjak mendekati nilai positif dan mulai konvergen.

Karena kami implementasi dengan sistem generational replacement penuh tanpa mekanisme elitism (population = new_population), kurva nilai objektif maksimum terlihat mengalami fluktuasi. Sebagai contoh, pada Run 2 benchmark standar, GA berhasil menemukan konfigurasi berkualitas tinggi dengan skor 440 pada generasi ke-78, namun pada generasi 150, nilai maksimum di populasi berada pada angka 410. Ini terjadi karena individu terbaik dapat terpecah kombinasi gennya akibat proses Single-Point Crossover atau mengalami mutasi acak.

Percobaan Lengkap 18-Run

Skenario	Konfigurasi Parameter	Run	Nilai Awal	Nilai Akhir	Peak (*Best*)	Status Validitas	Durasi (detik)	Rata-rata Akhir *(Peak)*	Rata-rata Durasi (s)
Eksperimen 1 (Kontrol : Populasi = 50)	Pop: 50 Iter: 50	1	345	450	450	Valid	0.1112	403.33 *(431.67)*	0.1036
		2	260	350	375	Valid	0.0916		
		3	330	410	420	Valid	0.1080		
	Pop: 50 Iter: 150	1	345	370	370	Valid	0.2359	375.00 *(403.33)*	0.2440
		2	260	410	440	Valid	0.2365		
		3	330	345	400	Valid	0.2596		
	Pop: 50 Iter: 300	1	345	460	480	Valid	0.7722	441.67 *(468.33)*	0.7410
		2	260	420	445	Valid	0.7141		
		3	330	445	480	Valid	0.7367		

Eksperimen 2 (Kontrol : Iterasi = 150)	Pop: 20 Iter: 150	1 2 3	345 260 330	380 355 390	410 405 425	Valid Valid Valid	0.1325 0.1384 0.1347	375.00 *(413.33)*	0.1352
	Pop: 50 Iter: 150	1 2 3	345 260 330	370 410 345	370 440 400	Valid Valid Valid	0.2359 0.2365 0.2596	375.00 *(403.33)*	0.2440
	Pop: 100 Iter: 150	1 2 3	345 260 330	445 435 450	480 465 480	Valid Valid Valid	0.6812 0.6985 0.6975	443.33 *(475.00)*	0.6924

Ketika ukuran populasi dikontrol tetap pada 50 individu, penambahan jumlah iterasi dari 50 menjadi 150 dan 300 memberikan ruang waktu yang lebih panjang bagi algoritma untuk melakukan eksplorasi dan eksploitasi blok-blok gen kendaraan yang menguntungkan.

Rata-rata nilai akhir meningkat secara konsisten: dari 403.33 (pada 50 iterasi) menjadi 441.67, dengan nilai puncak tertinggi mencapai 480. Namun, penambahan iterasi berbanding lurus dengan wdurasi, di mana durasi eksekusi meningkat dari 0.1036 detik pada 50 iterasi menjadi 0.7410 detik pada 300 iterasi.

Ketika jumlah iterasi dikontrol tetap pada 150 generasi, peningkatan ukuran populasi dari 20 menjadi 50 dan 100 memberikan dampak paling nyata terhadap stabilitas dan daya jangkau algoritma.

Pada populasi kecil 20 individu, ruang pencarian yang dieksplorasi sangat sempit sehingga populasi rentan mengalami premature convergence pada local optima dengan rata-rata nilai akhir hanya 375.

Sebaliknya, pada populasi besar 100 individu, keanekaragaman genetik terjaga dengan baik. Algoritma menghasilkan rata-rata nilai akhir tertinggi 443.33 dan nilai puncak rata-rata 475.00, dengan variasi hasil antar-run yang sangat rapat. Konsekuensinya, waktu eksekusi meningkat menjadi 0.6924 detik karena pada setiap generasi terdapat lebih banyak evaluasi individu. $100 \times 150 = 15.000$ evaluasi state.

d. Analisis

Untuk melakukan evaluasi komparatif yang adil, ketiga algoritma diuji menggunakan seed initial state yang sama persis: 42, 100, 2026. Batas solusi optimal pada studi kasus kapal kargo ini bernilai 485 dengan pemanfaatan kapasitas beban penuh $50/50$ unit.

Algoritma	Run 1 (Seed 42)	Run 2 (Seed 100)	Run 3 (Seed 2026)	Rata-rata Nilai Akhir	Nilai Terbaik (Peak)	Rata-rata Durasi (detik)
Steepest Ascent Hill-Climbing	440	475	455	456.67	475	0.0181 s
Simulated Annealing	475 (Best: 475)	475 (Best: 480)	485 (Best: 485)	478.33	485	0.0956 s
Genetic Algorithm (Pop=50, Iter=150)	370 (Best: 370)	410 (Best: 440)	345 (Best: 400)	375.00	440	0.2440 s

Simulated Annealing menjadi algoritma yang paling unggul karena berhasil menyentuh nilai global optimum absolut sebesar 485 dengan kapasitas muatan penuh 50/50 unit, dengan probabilitas Boltzmann ($e^{\Delta E/T}$) di temperatur tinggi yang

memungkinkannya menerima langkah memburuk sementara demi meloloskan diri dari kuncian penataan lokal. Sebaliknya, Steepest Ascent Hill-Climbing selalu terhenti di bawah optimum global pada rentang skor 440 hingga 475 karena sifatnya yang murni greedy membuatnya langsung berhenti begitu 50 sampel tetangga di sekitarnya tidak memberikan perbaikan nilai secara instan. Genetic Algorithm mencapai nilai puncak sebesar 440 pada populasi 50 dan hingga angka 480 pada populasi 100, namun kesulitan mencapai angka 485 secara konsisten akibat kelemahan operator Single-Point Crossover linier yang kerap memicu overlap spasial pada individu anak sehingga memerlukan generasi mutasi tambahan untuk memulihkan validitasnya.

Perbandingan hasil akhir memperlihatkan keunggulan konsisten SA atas SAHC dan GA. SAHC sering kali tertahan pada kapasitas muatan 46-48 unit karena menolak langkah penurunan nilai sementara yang sebenarnya dibutuhkan untuk menukar susunan kendaraan, sedangkan SA mampu mengeksekusi transisi tersebut secara fleksibel. Jika dibandingkan dengan GA, SA dan SAHC memiliki stabilitas nilai akhir yang lebih tinggi pada level individu karena bekerja secara trajektori tunggal, sedangkan GA yang menerapkan penggantian generational replacement kanonik tanpa mekanisme elitisme menyebabkan individu terbaik yang telah ditemukan berisiko terdegradasi pada generasi terakhir akibat pemotongan rekombinasi kromosom atau mutasi acak.

Ditinjau dari efisiensi waktu eksekusi, SAHC merupakan algoritma yang paling cepat dengan rata-rata durasi hanya 0,0181 detik karena proses pencariannya langsung konvergen dan berhenti di puncak lokal pertama hanya dalam 6 hingga 10 iterasi. SA menempati posisi menengah dengan durasi rata-rata 0,0956 detik karena mengevaluasi tepat satu tetangga di setiap iterasi sepanjang 2.000 langkah pendinginan tetap. Adapun GA membutuhkan waktu komputasi paling lama, yaitu 0,2440 detik pada 150 generasi dan melonjak hingga 0,7410 detik pada 300 generasi, sebagai konsekuensi dari pendekatan berbasis populasi paralel yang mengharuskan evaluasi fungsi objektif, seleksi roda rolet, dan rekombinasi bagi ribuan individu di setiap siklus. Evaluasi pada populasi 50 dan 150 generasi.

Simulated Annealing menunjukkan konsistensi paling prima dengan variansi hasil akhir yang sangat sempit pada rentang skor optimal, membuktikan bahwa laju pendinginan geometrik memberikan keseimbangan eksplorasi dan eksploitasi yang kokoh terlepas dari seberapa acak kondisi awalnya. SAHC konsistensi tingkat menengah pada rentang 440-475 karena kualitas solusinya sangat bergantung pada bentuk topologi cekungan di sekitar initial state acak yang dibangkitkan. Konsistensi GA sangat dipengaruhi oleh ukuran populasinya, di mana pada populasi 50 fluktuasi antargenerasi cukup lebar akibat tidak adanya elitisme, namun pada populasi 100 konsistensinya

meningkat drastis dengan hasil yang sangat stabil dan rapat pada kisaran skor 435 hingga 450.

Berdasarkan eksperimen 18 runs, peningkatan jumlah iterasi dari 50 ke 150 dan 300 generasi pada populasi tetap 50 terbukti mematangkan proses rekombinasi blok gen kendaraan, yang ditandai dengan kenaikan rata-rata skor akhir dari 403,33 menjadi 441,67 dan nilai puncak mencapai 480 dengan konsekuensi kenaikan durasi linier dari 0,10 detik ke 0,74 detik. Di sisi lain, peningkatan ukuran populasi dari 20 ke 50 dan 100 pada iterasi tetap 150 berperan krusial dalam menjaga keanekaragaman genetik guna mencegah premature convergence, di mana populasi kecil cepat mandek pada rata-rata skor 375,00 sedangkan populasi besar mampu menjelajahi ruang pencarian secara luas hingga menghasilkan rata-rata skor akhir tertinggi sebesar 443,33. Secara keseluruhan, ukuran populasi menentukan kualitas dan luasnya ruang eksplorasi, sementara jumlah iterasi menentukan kedalaman eksploitasi menuju solusi optimal.

Skema penalti bertingkat pada fungsi evaluasi terbukti memberikan gradien pemandu yang sangat efektif bagi ketiga algoritma dalam memulihkan initial state yang semula bernilai negatif ekstrem menjadi solusi yang 100% valid. SAHC memanfaatkan gradien penalti ini untuk mengeliminasi pelanggaran batasan hanya dalam kurang dari empat iterasi awal, sementara pada GA formula pergeseran skala linier berhasil mempertahankan disparitas kualitas antar-individu di domain positif sehingga kromosom yang hanya melanggar sedikit sel tetap medmiliki probabilitas seleksi yang jauh lebih tinggi untuk bereproduksi dibanding kromosom yang rusak parah.

BAB 4
KESIMPULAN DAN SARAN

4.1. Kesimpulan

Permasalahan optimasi penataan penempatan muatan kendaraan pada geladak kapal kargo PT Owen Intim Line telah berhasil dimodelkan dan diselesaikan secara efektif menggunakan pendekatan Local Search. Representasi terpadu ShipState dengan sistem koordinat grid berbasis titik acuan 0,0 pada sudut kiri bawah kapal, didukung fungsi objektif berpenalti bertingkat dan tiga operator langkah. terbukti mampu memfasilitasi pencarian solusi yang valid dan optimal bagi ketiga algoritma yang diuji.

Berdasarkan seluruh hasil pengujian dan analisis perbandingan, Simulated Annealing dinobatkan sebagai algoritma terbaik secara keseluruhan. Algoritma ini menunjukkan konsistensi tertinggi dan menjadi satu-satunya metode yang berhasil mencapai solusi optimal global absolut dengan nilai fungsi objektif 485 serta pemanfaatan kapasitas beban penuh 50/50 unit, dalam waktu komputasi yang sangat efisien yakni 0,0956 detik. Keunggulan tersebut bersumber dari mekanisme penerimaan langkah memburuk probabilistik yang mencegah algoritma terperangkap pada puncak lokal di awal proses pendinginan.

Di sisi lain, Steepest Ascent Hill-Climbing unggul mutlak dalam aspek kecepatan eksekusi dengan rata-rata durasi hanya 0,0181 detik dan mampu menghasilkan solusi yang memadai pada rentang 440 hingga 475, meskipun sifat pencariannya yang murni greedy membuatnya rentan mengalami terminasi prematur pada local optimum pertama yang ditemui. Genetic Algorithm membuktikan keandalannya dalam penjelajahan ruang solusi paralel berskala luas dan mampu mencatatkan nilai puncak hingga 480 pada populasi 100. Namun, performa nilai akhir GA sangat bergantung pada parameter ukuran populasi dan jumlah iterasi, serta memerlukan durasi komputasi yang paling tinggi sebagai konsekuensi dari evaluasi multi-individu di setiap generasinya.

4.2. Saran

Untuk meningkatkan performa dan keandalan sistem pencarian solusi di masa mendatang, disarankan untuk mengintegrasikan mekanisme elitisme ke dalam implementasi Genetic Algorithm. Dengan menyalin secara langsung satu atau dua individu terbaik dari populasi saat ini ke generasi berikutnya tanpa melalui proses crossover dan mutasi, konfigurasi berkualitas tinggi yang telah berhasil ditemukan tidak akan terdegradasi oleh fluktuasi acak pada iterasi-iterasi akhir evolusi.

Selain itu, operator Single-Point Crossover linier pada GA sebaiknya dikembangkan menjadi operator 2D Geometric Crossover. Melalui pendekatan pemotongan geometris grid geladak kapal, rekombinasi informasi spasial antara dua kapal induk dapat dilakukan berdasarkan zona fisik kapal , sehingga risiko terjadinya overlap antar-kendaraan pada individu anak dapat diminimalkan secara signifikan.

Saran berikutnya adalah mengkaji penerapan algoritma hibrida (Memetic Algorithm) yang memadukan keunggulan eksplorasi global Genetic Algorithm dengan efisiensi eksploitasi lokal Steepest Ascent Hill-Climbing. Pada skema ini, setiap individu baru hasil reproduksi diperbaiki terlebih dahulu posisinya menggunakan beberapa langkah hill-climbing sebelum dimasukkan ke dalam populasi, sehingga konvergensi menuju solusi berkualitas tinggi dapat tercapai jauh lebih cepat. Terakhir, pada algoritma Simulated Annealing, disarankan untuk menguji jadwal pendinginan adaptif di mana laju penurunan temperatur disesuaikan secara dinamis berdasarkan variansi nilai objektif terkini, guna meningkatkan efisiensi komputasi pada kasus uji berskala lebih besar.

PEMBAGIAN TUGAS

Nama	NIM	Pembagian Tugas
Christian Wilfredo Pakpahan	18224038	Implementasi: - Genetic Algorithm Laporan: - Deskripsi persoalan - 2.2.c Pembahasan - 3.3 Hasil - 3.4 Analisis - 4.1 Kesimpulan - 4.2 Saran
Nicholas Putra Halim	18224098	Implementasi: - Simulated Annealing Laporan: - 2.2.b Pembahasan - 3.2 Hasil
Brandon Zeko Alexander	18224118	Implementasi: - Hill Climbing Steepest Ascent Laporan: BAB 2 PEMBAHASAN ● 2.1 Objective Function ● 2.2 Implementasi Algoritma Local Search (a. Hill-Climbing Steepest Ascent)

REFERENSI

1. Slide Kuliah IF3070 - Dasar Inteligensi Artifisial 2026. Program Studi Teknik Informatika, Institut Teknologi Bandung.
2. Kothari, S. (2023). *Local Search Algorithms in AI: A Comprehensive Guide.* Simplilearn. Diakses dari: https://www.simplilearn.com/local-search-algorithms-in-ai-article
3. Scaler. *Types of Local Search Algorithm.* Scaler Topics. Diakses dari: https://www.scaler.com/topics/artificial-intelligence-tutorial/types-of-local-search-algorithm
4. Quora. (2023). *Why do algorithms become entrapped in local optima?* Diakses dari: https://www.quora.com/Why-do-algorithms-become-entrapped-in-local-optima
5. GeeksforGeeks. (2024). *Genetic Algorithms – Introduction to Evolutionary Computation.* Diakses dari: https://www.geeksforgeeks.org/genetic-algorithms/
6. Medium. The Steepest Ascent Hill Climbing Algorithm Unraveled. Mirko Peters https://pub.towardsai.net/the-steepest-ascent-hill-climbing-algorithm-unraveled-32535e612289
7. Baeldung. *Simulated Annealing.* Baeldung | Computer Science. Diakses dari: https://www.baeldung.com/cs/simulated-annealing
8. Wikipedia. Hill climbing. Wikipedia. Diakses dari: https://en.wikipedia.org/wiki/Hill_climbing
9. Wikipedia. *Simulated Annealing*. Wikipedia. Diakses dari: https://en.wikipedia.org/wiki/Simulated_annealing
10. Wikipedia. *Genetic Algorithm.* Wikipedia. Diakses dari: https://en.wikipedia.org/wiki/Genetic_algorithm

LAMPIRAN FORM PENGGUNAAN GENERATIVE AI

Dengan ini, kami:
Nama/NIM :

- Christian Wilfredo Pakpahan/18224038
- Nicholas Putra Halim/18224098
- Brandon Zeko Alexander/18224118

Fakultas/Sekolah : Sekolah Teknik Elektro & Informatika - Komputasi
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Intelegensi Artifisial
Judul Tugas/Proposal : Optimasi Penataan Kendaraan pada Kapal Menggunakan Local Search

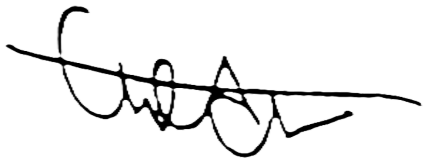
Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	3
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Christian Wilfredo Pakpahan

18224038